NIET Greater Noida

Intelligent Medical Inventory Tracking System

Source Code Listing

Object Oriented Techniques using Java (CCSE0355)
Semester III | B.Tech CSE (AI) | Capstone Project
Faculty: Ms Shweta Singh, Ms Barkha Bhardwaj

Name	Roll No.
Devvrat Tripathi	0251CSAI231
Manas Srivastava	0251CSAI144
Pranjal Sharma	0251CSAI141
Hardik Kumar	0251CSAI195
Mritunjay Gupta	0251CSAI132

Group 24

Java source files	44
Lines of code	2,224
Packages	8
Build	JDK only, no external libraries
Repository	github.com/DevvTrip/Intelligent-Medical-Inventory-Tracking-System

Submitted 28 October 2026

Contents

Package model 4

src/model/Hospital.java 4

src/model/Injection.java 4

src/model/Medicine.java 4

src/model/Supplier.java 6

src/model/Syrup.java 6

src/model/Tablet.java 6

Package usermanagement 7

src/usermanagement/LoginCheck.java 7

src/usermanagement/User.java 7

Package businesslogic 7

src/businesslogic/DataStore.java 7

src/businesslogic/DuplicateMedicineException.java 8

src/businesslogic/ExpiredMedicineException.java 8

src/businesslogic/ExpiryMonitor.java 8

src/businesslogic/InvalidInputException.java 9

src/businesslogic/Inventory.java 9

src/businesslogic/MedicineValidation.java 10

src/businesslogic/OutOfStockException.java 11

src/businesslogic/Repository.java 11

src/businesslogic/StockService.java 11

Package reporting 12

src/reporting/Alertable.java 12

src/reporting/AnalyticsReport.java 12

src/reporting/ReportMaker.java 13

src/reporting/Reportable.java 13

Package dataprocessing 13

src/dataprocessing/SalesData.java 13

src/dataprocessing/SampleData.java 14

Package utils 15

src/utils/BackupUtil.java 15

src/utils/FileUtil.java 15

src/utils/InventoryFileLock.java 17

src/utils/SortUtil.java 17

Package network **18**

src/network/CounterClient.java 18

src/network/InventoryServer.java 18

Package gui **19**

src/gui/ConsoleMenu.java 19

src/gui/Dashboard.java 20

src/gui/DashboardActions.java 22

src/gui/LoginScreen.java 23

src/gui/Main.java 24

src/gui/MedicineForm.java 24

src/gui/MedicineFormActions.java 26

src/gui/ReportScreen.java 26

src/gui/SearchScreen.java 28

src/gui/SellActions.java 29

src/gui/SellDialog.java 30

src/gui/StatusRenderer.java 31

src/gui/SummaryCards.java 31

src/gui/Ui.java 32

Appendix: build scripts **33**

run.bat 33

run.sh 33

Package model

Domain classes: medicine hierarchy, supplier and hospital

src/model/Hospital.java

```
package model;

import businesslogic.Inventory;

public class Hospital {

    private String name;
    private Inventory inventory;

    public Hospital(String name) {
        this.name = name;
        this.inventory = new Inventory();
    }

    public String getName() { return name; }
    public Inventory getInventory() { return inventory; }
}
```

src/model/Injection.java

```
package model;

import java.time.LocalDate;

public class Injection extends Medicine {

    private int doseMg;

    public Injection(String id, String name, double price, int qty,
            LocalDate expiryDate, int threshold, Supplier supplier, int doseMg) {
        super(id, name, price, qty, expiryDate, threshold, supplier);
        this.doseMg = doseMg;
    }

    public int getExtra() {
        return doseMg;
    }

    public String getStorageInfo() {
        return "Injection vial of " + doseMg + " mg, store in a cold place";
    }

    // cold storage ka extra 15 rupees charge add kiya hai
    public double calculatePrice() {
        return getPrice() + 15.0;
    }
}
```

src/model/Medicine.java

```
package model;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import reporting.Alertable;
import reporting.Reportable;

public abstract class Medicine implements Alertable, Reportable, Comparable<Medicine> {

    private String id;
    private String name;
    private double price;
    private int qty;
    private LocalDate expiryDate;
    private int threshold;
    private Supplier supplier;

    public Medicine(String id, String name, double price, int qty,
            LocalDate expiryDate, int threshold, Supplier supplier) {
        this.id = id;
        this.name = name;
        this.price = price;
        this.qty = qty;
        this.expiryDate = expiryDate;
        this.threshold = threshold;
        this.supplier = supplier;
    }

    public String getId() { return id; }
```

```
    public String getName() { return name; }
    public double getPrice() { return price; }
    public int getQty() { return qty; }
    public LocalDate getExpiryDate() { return expiryDate; }
    public int getThreshold() { return threshold; }
    public Supplier getSupplier() { return supplier; }

    public void setQty(int qty) { this.qty = qty; }
    public void setPrice(double price) { this.price = price; }
    public void setExpiryDate(LocalDate expiryDate) { this.expiryDate = expiryDate; }

    public abstract String getStorageInfo();
    public abstract int getExtra();

    public double calculatePrice() {
        return price;
    }

    public boolean isExpired() {
        return expiryDate.isBefore(LocalDate.now());
    }

    public boolean isExpiringSoon() {
        long days = ChronoUnit.DAYS.between(LocalDate.now(), expiryDate);
        return days >= 0 && days <= 30;
    }

    public boolean isLowStock() {
        return qty < threshold;
    }

    public String getStatus() {
        if (isExpired()) {
            return "EXPIRED";
        }
        if (isExpiringSoon() && isLowStock()) {
            return "EXPIRING SOON + LOW STOCK";
        }
        if (isExpiringSoon()) {
            return "EXPIRING SOON";
        }
        if (isLowStock()) {
            return "LOW STOCK";
        }
        return "OK";
    }

    public String getReportLine() {
        return id + " | " + name + " | qty " + qty + " | " + expiryDate + " | " + getStatus();
    }

    public String toCsv() {
        StringBuilder sb = new StringBuilder();
        sb.append(getClass().getSimpleName()).append(",");
        sb.append(id).append(",");
        sb.append(name).append(",");
        sb.append(price).append(",");
        sb.append(qty).append(",");
        sb.append(expiryDate).append(",");
        sb.append(threshold).append(",");
        sb.append(supplier.getName()).append(",");
        sb.append(supplier.getPhone()).append(",");
        sb.append(getExtra());
        return sb.toString();
    }

    // agar expiry same hai to id se compare karo, warna TreeSet ek medicine ko hata deta hai
    public int compareTo(Medicine other) {
        int result = expiryDate.compareTo(other.expiryDate);
        if (result == 0) {
            result = id.compareTo(other.id);
        }
        return result;
    }

    public String toString() {
        return id + " - " + name + " qty " + qty;
    }
}
```

src/model/Supplier.java

```java
package model;

public class Supplier {

    private String name;
    private String phone;

    public Supplier(String name, String phone) {
        this.name = name;
        this.phone = phone;
    }

    public String getName() { return name; }
    public String getPhone() { return phone; }

    public boolean equals(Object obj) {
        if (this == obj) {
            return true;
        }
        if (!(obj instanceof Supplier)) {
            return false;
        }
        Supplier other = (Supplier) obj;
        return name.equalsIgnoreCase(other.name);
    }

    public int hashCode() {
        return name.toLowerCase().hashCode();
    }

    public String toString() {
        return name + " (" + phone + ")";
    }
}
```

src/model/Syrup.java

```java
package model;

import java.time.LocalDate;

public class Syrup extends Medicine {

    private int bottleMl;

    public Syrup(String id, String name, double price, int qty,
            LocalDate expiryDate, int threshold, Supplier supplier, int bottleMl) {
        super(id, name, price, qty, expiryDate, threshold, supplier);
        this.bottleMl = bottleMl;
    }

    public int getExtra() {
        return bottleMl;
    }

    public String getStorageInfo() {
        return "Syrup bottle of " + bottleMl + " ml, keep away from sunlight";
    }

    // bottle ka charge 5% ke hisaab se add kiya hai
    public double calculatePrice() {
        return getPrice() * 1.05;
    }
}
```

src/model/Tablet.java

```java
package model;

import java.time.LocalDate;

public class Tablet extends Medicine {

    private int tabletsPerStrip;

    public Tablet(String id, String name, double price, int qty,
            LocalDate expiryDate, int threshold, Supplier supplier, int tabletsPerStrip) {
        super(id, name, price, qty, expiryDate, threshold, supplier);
        this.tabletsPerStrip = tabletsPerStrip;
    }

    public int getExtra() {
        return tabletsPerStrip;
```

```
    }

    public String getStorageInfo() {
        return "Tablet strip of " + tabletsPerStrip + " tablets, keep in a cool dry place";
    }

    // packing ka 2 rupees ka charge price me add ho jata hai
    public double calculatePrice() {
        return getPrice() + 2.0;
    }
}
```

Package usermanagement

User record and login check

src/usermanagement/LoginCheck.java

```
package usermanagement;

public class LoginCheck {

    public static User check(String username, String password) {
        if (username == null || password == null) {
            return null;
        }
        String user = username.trim().toLowerCase();
        if (user.equals("admin") && password.equals("admin123")) {
            return new User("admin", "ADMIN");
        }
        if (user.equals("pharma") && password.equals("pharma123")) {
            return new User("pharma", "PHARMACIST");
        }
        return null;
    }
}
```

src/usermanagement/User.java

```
package usermanagement;

public class User {

    private String username;
    private String role;

    public User(String username, String role) {
        this.username = username;
        this.role = role;
    }

    public String getUsername() { return username; }
    public String getRole() { return role; }

    public boolean isAdmin() {
        return role.equals("ADMIN");
    }
}
```

Package businesslogic

Inventory, stock operations, persistence and expiry monitoring

src/businesslogic/DataStore.java

```
package businesslogic;

import java.util.ArrayList;

public class DataStore<T> implements Repository<T> {

    private ArrayList<T> items = new ArrayList<T>();

    public void add(T item) {
        items.add(item);
    }

    public void remove(T item) {
        items.remove(item);
    }

    public ArrayList<T> getAll() {
        return items;
    }
```

```
20
21      public int size() {
22          return items.size();
23      }
24
25      public void clear() {
26          items.clear();
27      }
28  }
```

src/businesslogic/DuplicateMedicineException.java

```
1   package businesslogic;
2
3   public class DuplicateMedicineException extends Exception {
4
5       public DuplicateMedicineException(String message) {
6           super(message);
7       }
8   }
```

src/businesslogic/ExpiredMedicineException.java

```
1   package businesslogic;
2
3   public class ExpiredMedicineException extends Exception {
4
5       public ExpiredMedicineException(String message) {
6           super(message);
7       }
8   }
```

src/businesslogic/ExpiryMonitor.java

```
1   package businesslogic;
2
3   import java.io.IOException;
4   import javax.swing.JLabel;
5   import javax.swing.SwingUtilities;
6   import model.Medicine;
7
8   public class ExpiryMonitor implements Runnable {
9       private Inventory inventory;
10      private JLabel statusLabel;
11      private Runnable refreshView;
12
13      public ExpiryMonitor(Inventory inventory, JLabel statusLabel, Runnable refreshView) {
14          this.inventory = inventory;
15          this.statusLabel = statusLabel;
16          this.refreshView = refreshView;
17      }
18
19      public void run() {
20          while (!Thread.currentThread().isInterrupted()) {
21              String status;
22              boolean refreshed = false;
23              try {
24                  inventory.refresh();
25                  int low = 0;
26                  int expiring = 0;
27                  for (Medicine medicine : inventory.getMedicines()) {
28                      if (medicine.isLowStock() && !medicine.isExpired()) {
29                          low++;
30                      }
31                      if (medicine.isExpiringSoon()) {
32                          expiring++;
33                      }
34                  }
35                  status = "Monitor: " + low + " low stock, " + expiring + " expiring in 30 days";
36                  refreshed = true;
37              } catch (IOException e) {
38                  status = "Could not refresh inventory: " + e.getMessage();
39              }
40              final String message = status;
41              final boolean updateTable = refreshed;
42              SwingUtilities.invokeLater(new Runnable() {
43                  public void run() {
44                      statusLabel.setText(message);
45                      if (updateTable) {
46                          refreshView.run();
47                      }
48                  }
49              });
50              try {
```

```
                Thread.sleep(5000);
            } catch (InterruptedException e) {
                Thread.currentThread().interrupt();
            }
        }
    }
}
```

src/businesslogic/InvalidInputException.java

```
package businesslogic;

public class InvalidInputException extends Exception {

    public InvalidInputException(String message) {
        super(message);
    }
}
```

src/businesslogic/Inventory.java

```
package businesslogic;

import java.io.File;
import java.io.IOException;
import java.time.LocalTime;
import java.util.ArrayList;
import java.util.HashMap;
import java.util.HashSet;
import java.util.LinkedList;
import java.util.TreeSet;
import model.Medicine;
import model.Supplier;
import utils.FileUtil;
import utils.InventoryFileLock;

public class Inventory {
    private DataStore<Medicine> store = new DataStore<Medicine>();
    private HashMap<String, Medicine> medMap = new HashMap<String, Medicine>();
    private HashSet<Supplier> supplierSet = new HashSet<Supplier>();
    private TreeSet<Medicine> expirySet = new TreeSet<Medicine>();
    private LinkedList<String> activityLog = new LinkedList<String>();
    private StockService stockService = new StockService(this);

    public void initialize(ArrayList<Medicine> sample) throws IOException {
        try (InventoryFileLock lock = InventoryFileLock.acquire()) {
            if (!new File(FileUtil.DATA_FILE).exists()) {
                FileUtil.saveMedicines(sample);
            }
            install(FileUtil.loadMedicines());
        }
    }

    public void refresh() throws IOException {
        try (InventoryFileLock lock = InventoryFileLock.acquire()) {
            install(FileUtil.loadMedicines());
        }
    }

    public void addMedicine(Medicine medicine)
            throws IOException, DuplicateMedicineException, InvalidInputException {
        MedicineValidation.check(medicine);
        try (InventoryFileLock lock = InventoryFileLock.acquire()) {
            ArrayList<Medicine> list = FileUtil.loadMedicines();
            for (Medicine item : list) {
                if (item.getId().equals(medicine.getId())) {
                    throw new DuplicateMedicineException("ID " + medicine.getId() + " already exists");
                }
            }
            list.add(medicine);
            saveAndUpdate(list, "added " + medicine.getName());
        }
    }

    public void updateMedicine(Medicine medicine)
            throws IOException, InvalidInputException {
        MedicineValidation.check(medicine);
        try (InventoryFileLock lock = InventoryFileLock.acquire()) {
            ArrayList<Medicine> list = FileUtil.loadMedicines();
            for (int i = 0; i < list.size(); i++) {
                if (list.get(i).getId().equals(medicine.getId())) {
                    list.set(i, medicine);
                    saveAndUpdate(list, "updated " + medicine.getName());
                    return;
                }
```

```
            }
            throw new InvalidInputException("Medicine no longer exists");
        }
    }

    public void deleteMedicine(String id) throws IOException, InvalidInputException {
        try (InventoryFileLock lock = InventoryFileLock.acquire()) {
            ArrayList<Medicine> list = FileUtil.loadMedicines();
            for (int i = 0; i < list.size(); i++) {
                if (list.get(i).getId().equals(id)) {
                    Medicine removed = list.remove(i);
                    saveAndUpdate(list, "deleted " + removed.getName());
                    return;
                }
            }
            throw new InvalidInputException("Medicine no longer exists");
        }
    }

    public synchronized ArrayList<Medicine> searchMedicine(String keyword) {
        ArrayList<Medicine> found = new ArrayList<Medicine>();
        String word = keyword.toLowerCase();
        for (Medicine medicine : store.getAll()) {
            if (medicine.getName().toLowerCase().contains(word) || medicine.getId().contains(word)) {
                found.add(medicine);
            }
        }
        return found;
    }
    public synchronized Medicine searchMedicine(int idNumber) {
        return medMap.get(String.valueOf(idNumber));
    }
    public synchronized Medicine getById(String id) { return medMap.get(id); }

    synchronized void saveAndUpdate(ArrayList<Medicine> list, String message) throws IOException {
        FileUtil.saveMedicines(list);
        install(list);
        activityLog.addFirst(LocalTime.now().withNano(0) + " " + message);
        if (activityLog.size() > 30) {
            activityLog.removeLast();
        }
    }
    private synchronized void install(ArrayList<Medicine> list) {
        store.clear();
        medMap.clear();
        expirySet.clear();
        supplierSet.clear();
        for (Medicine medicine : list) {
            store.add(medicine);
            medMap.put(medicine.getId(), medicine);
            expirySet.add(medicine);
            supplierSet.add(medicine.getSupplier());
        }
    }
    public synchronized ArrayList<Medicine> getMedicines() { return new ArrayList<Medicine>(store.getAll()); }
    public synchronized ArrayList<Medicine> getByExpiryOrder() { return new ArrayList<Medicine>(expirySet); }
    public synchronized ArrayList<Supplier> getSupplierList() { return new ArrayList<Supplier>(supplierSet); }
    public synchronized LinkedList<String> getActivityLog() { return new LinkedList<String>(activityLog); }
    public StockService getStockService() { return stockService; }
}
```

src/businesslogic/MedicineValidation.java

```
package businesslogic;

import model.Medicine;

public class MedicineValidation {
    public static void check(Medicine medicine) throws InvalidInputException {
        try {
            if (Integer.parseInt(medicine.getId()) <= 0) {
                throw new InvalidInputException("ID must be a positive number");
            }
        } catch (NumberFormatException e) {
            throw new InvalidInputException("ID must be a positive number");
        }
        if (medicine.getName().trim().isEmpty()
                || medicine.getSupplier().getName().trim().isEmpty()) {
            throw new InvalidInputException("Medicine name and supplier name cannot be empty");
        }
        if (medicine.getPrice() < 0 || medicine.getQty() < 0
                || medicine.getThreshold() < 1 || medicine.getExtra() < 1) {
            throw new InvalidInputException("Use a valid price, non-negative quantity and positive
    limit/extra");
        }
```

```
        // comma allowed nahi hai, kyunki CSV me comma se hi fields alag hote hain
        String[] text = {medicine.getName(), medicine.getSupplier().getName(),
                medicine.getSupplier().getPhone()};
        for (String value : text) {
            if (value.contains(",")) {
                throw new InvalidInputException("Do not use commas in text fields");
            }
        }
    }
}
```

src/businesslogic/OutOfStockException.java

```
package businesslogic;

public class OutOfStockException extends Exception {

    public OutOfStockException(String message) {
        super(message);
    }
}
```

src/businesslogic/Repository.java

```
package businesslogic;

import java.util.ArrayList;

public interface Repository<T> {
    void add(T item);
    void remove(T item);
    ArrayList<T> getAll();
    int size();
}
```

src/businesslogic/StockService.java

```
package businesslogic;

import java.io.IOException;
import java.util.ArrayList;
import java.util.HashMap;
import model.Medicine;
import utils.FileUtil;
import utils.InventoryFileLock;

public class StockService {
    private Inventory inventory;
    private HashMap<String, Integer> soldCount = new HashMap<String, Integer>();

    StockService(Inventory inventory) {
        this.inventory = inventory;
    }

    public synchronized int sellMedicine(String id, int qty)
            throws IOException, OutOfStockException, ExpiredMedicineException, InvalidInputException {
        if (qty <= 0) {
            throw new InvalidInputException("Quantity must be more than 0");
        }
        // lock ke andar fresh data load karo, warna server ka sale miss ho jata hai
        try (InventoryFileLock lock = InventoryFileLock.acquire()) {
            ArrayList<Medicine> list = FileUtil.loadMedicines();
            for (Medicine medicine : list) {
                if (medicine.getId().equals(id)) {
                    if (medicine.isExpired()) {
                        throw new ExpiredMedicineException(medicine.getName() + " is expired");
                    }
                    if (medicine.getQty() < qty) {
                        throw new OutOfStockException("Not enough stock, only " + medicine.getQty() + "
    left");
                    }
                    medicine.setQty(medicine.getQty() - qty);
                    inventory.saveAndUpdate(list, "sold " + qty + " of " + medicine.getName());
                    recordSale(id, qty);
                    return medicine.getQty();
                }
            }
            throw new InvalidInputException("Medicine not found: " + id);
        }
    }

    public synchronized void recordSale(String id, int qty) {
        Integer old = soldCount.get(id);
        if (old == null) {
            soldCount.put(id, qty);
```

```
48          } else {
49              soldCount.put(id, old + qty);
50          }
51      }
52
53      public synchronized HashMap<String, Integer> getSoldCount() {
54          return new HashMap<String, Integer>(soldCount);
55      }
56  }
```

Package reporting

Report interfaces and report text generation

src/reporting/Alertable.java

```
1   package reporting;
2
3   public interface Alertable {
4       boolean isLowStock();
5       boolean isExpired();
6       boolean isExpiringSoon();
7       String getStatus();
8   }
```

src/reporting/AnalyticsReport.java

```
1   package reporting;
2
3   import java.util.ArrayList;
4   import java.util.HashMap;
5   import businesslogic.Inventory;
6   import model.Medicine;
7   import model.Supplier;
8
9   public class AnalyticsReport {
10
11      public static String fastMovingReport(Inventory inventory) {
12          StringBuffer sb = new StringBuffer();
13          sb.append("FAST MOVING MEDICINES\n\n");
14          HashMap<String, Integer> sold = inventory.getStockService().getSoldCount();
15          ArrayList<String> ids = new ArrayList<String>(sold.keySet());
16          ids.sort((id1, id2) -> sold.get(id2) - sold.get(id1));
17          if (ids.isEmpty()) {
18              sb.append("No sales yet");
19          }
20          for (String id : ids) {
21              Medicine m = inventory.getById(id);
22              String name = "removed medicine";
23              if (m != null) {
24                  name = m.getName();
25              }
26              sb.append(name).append(" | sold ").append(sold.get(id)).append("\n");
27          }
28          return sb.toString();
29      }
30
31      public static String supplierReport(Inventory inventory) {
32          StringBuffer sb = new StringBuffer();
33          sb.append("SUPPLIER LIST\n\n");
34          for (Supplier s : inventory.getSupplierList()) {
35              sb.append(s.toString()).append("\n");
36          }
37          return sb.toString();
38      }
39
40      public static String activityReport(Inventory inventory) {
41          StringBuffer sb = new StringBuffer();
42          sb.append("RECENT ACTIVITY\n\n");
43          for (String line : inventory.getActivityLog()) {
44              sb.append(line).append("\n");
45          }
46          return sb.toString();
47      }
48  }
```

src/reporting/ReportMaker.java

```
package reporting;

import java.time.LocalDate;
import java.time.temporal.ChronoUnit;
import java.util.ArrayList;
import businesslogic.Inventory;
import model.Medicine;
import utils.SortUtil;

public class ReportMaker {

    public static String stockReport(Inventory inventory) {
        StringBuilder sb = new StringBuilder();
        sb.append("STOCK REPORT\n\n");
        double totalValue = 0;
        int count = 0;
        for (Medicine m : inventory.getMedicines()) {
            sb.append(m.getReportLine()).append("\n");
            sb.append("   ").append(m.getStorageInfo()).append("\n");
            totalValue = totalValue + m.getQty() * m.calculatePrice();
            count++;
        }
        sb.append("\nTotal medicines: ").append(count);
        sb.append("\nTotal stock value: Rs ").append(String.format("%.2f", totalValue));
        return sb.toString();
    }

    public static String expiryReport(Inventory inventory) {
        StringBuilder sb = new StringBuilder();
        sb.append("EXPIRY REPORT (nearest date first)\n\n");
        ArrayList<Medicine> list = inventory.getByExpiryOrder();
        for (Medicine m : list) {
            long days = ChronoUnit.DAYS.between(LocalDate.now(), m.getExpiryDate());
            sb.append(m.getReportLine()).append(" | days left ").append(days).append("\n");
        }
        return sb.toString();
    }

    public static String reorderReport(Inventory inventory) {
        StringBuffer sb = new StringBuffer();
        sb.append("REORDER REPORT\n\n");
        ArrayList<Medicine> lowList = SortUtil.filterLowStock(inventory.getMedicines());
        if (lowList.isEmpty()) {
            sb.append("Nothing to reorder");
        }
        for (Medicine m : lowList) {
            // itna order karo ki stock low-stock limit ka double ho jaye
            int qtyToOrder = m.getThreshold() * 2 - m.getQty();
            sb.append(m.getName()).append(" | qty ").append(m.getQty());
            sb.append(" | limit ").append(m.getThreshold());
            sb.append(" | order ").append(qtyToOrder).append("\n");
        }
        return sb.toString();
    }
}
```

src/reporting/Reportable.java

```
package reporting;

public interface Reportable {
    String getReportLine();
}
```

Package dataprocessing

Sample data and sales history

src/dataprocessing/SalesData.java

```
package dataprocessing;

public class SalesData {

    private int[][] weeklySales = new int[4][];

    public SalesData() {
        weeklySales[0] = new int[] {12, 8, 15};
        weeklySales[1] = new int[] {6, 4};
        weeklySales[2] = new int[] {3, 5, 2, 4};
        weeklySales[3] = new int[] {1};
    }

```

```
    public int getCategoryTotal(int category) {
        int total = 0;
        for (int i = 0; i < weeklySales[category].length; i++) {
            total = total + weeklySales[category][i];
        }
        return total;
    }

    public int getGrandTotal() {
        int total = 0;
        for (int i = 0; i < weeklySales.length; i++) {
            total = total + getCategoryTotal(i);
        }
        return total;
    }

    public String getReport() {
        String[] names = {"Tablet", "Syrup", "Injection", "Other"};
        StringBuilder sb = new StringBuilder();
        sb.append("WEEKLY SALES SAMPLE (jagged array)\n\n");
        for (int i = 0; i < weeklySales.length; i++) {
            sb.append(names[i]);
            sb.append(" | days ").append(weeklySales[i].length);
            sb.append(" | total ").append(getCategoryTotal(i)).append("\n");
        }
        sb.append("\nGrand total: ").append(getGrandTotal());
        return sb.toString();
    }
}
```

src/dataprocessing/SampleData.java

```
package dataprocessing;

import java.time.LocalDate;
import java.util.ArrayList;
import model.Injection;
import model.Medicine;
import model.Supplier;
import model.Syrup;
import model.Tablet;

public class SampleData {

    // dates aaj ki date se banayi hain, taaki expiry warning demo me hamesha dikhe
    public static ArrayList<Medicine> getMedicines() {
        ArrayList<Medicine> list = new ArrayList<Medicine>();
        LocalDate today = LocalDate.now();

        Supplier s1 = new Supplier("MedPlus Distributors", "9810011111");
        Supplier s2 = new Supplier("HealthCare Supplies", "9820022222");
        Supplier s3 = new Supplier("Sun Pharma Depot", "9830033333");

        list.add(new Tablet("101", "Paracetamol 500", 22.5, 120,
                today.plusMonths(10), 30, s1, 10));
        list.add(new Tablet("102", "Amoxicillin 250", 45.0, 18,
                today.plusMonths(6), 25, s2, 10));
        list.add(new Syrup("103", "Cough Syrup Benadryl", 95.0, 40,
                today.plusDays(10), 15, s1, 100));
        list.add(new Injection("104", "Tetanus Toxoid", 150.0, 12,
                today.plusMonths(11), 10, s3, 500));
        list.add(new Tablet("105", "Metformin 500", 35.0, 8,
                today.plusMonths(4), 20, s2, 15));
        list.add(new Syrup("106", "ORS Solution", 40.0, 60,
                today.minusDays(10), 20, s3, 200));
        list.add(new Injection("107", "Insulin Injection", 320.0, 6,
                today.plusDays(20), 10, s1, 100));
        list.add(new Tablet("108", "Cetirizine 10", 15.0, 150,
                today.plusMonths(14), 40, s2, 10));
        list.add(new Syrup("109", "Antacid Gel", 70.0, 9,
                today.plusMonths(3), 12, s3, 170));
        list.add(new Injection("110", "Dexamethasone", 60.0, 25,
                today.minusDays(45), 10, s1, 4));
        return list;
    }
}
```

Package utils

File helpers, backup, sorting and file locking

src/utils/BackupUtil.java

```
package utils;

import java.io.File;
import java.io.FileInputStream;
import java.io.FileOutputStream;
import java.io.IOException;

public class BackupUtil {
    public static final String BACKUP_FILE = "backup/medicines_backup.csv";

    public static void backup() throws IOException {
        try (InventoryFileLock lock = InventoryFileLock.acquire()) {
            FileUtil.loadMedicines();
            copyFile(FileUtil.DATA_FILE, BACKUP_FILE);
        }
    }

    public static void restore() throws IOException {
        try (InventoryFileLock lock = InventoryFileLock.acquire()) {
            FileUtil.loadMedicines(BACKUP_FILE);
            copyFile(BACKUP_FILE, FileUtil.DATA_FILE);
        }
    }

    private static void copyFile(String source, String target) throws IOException {
        new File(target).getParentFile().mkdirs();
        try (FileInputStream input = new FileInputStream(source);
                FileOutputStream output = new FileOutputStream(target)) {
            byte[] buffer = new byte[1024];
            int length;
            while ((length = input.read(buffer)) != -1) {
                output.write(buffer, 0, length);
            }
        }
    }
}
```

src/utils/FileUtil.java

```
package utils;

import java.io.BufferedReader;
import java.io.File;
import java.io.FileReader;
import java.io.FileWriter;
import java.io.IOException;
import java.time.LocalDate;
import java.time.format.DateTimeParseException;
import java.util.ArrayList;
import java.util.HashSet;
import businesslogic.InvalidInputException;
import businesslogic.MedicineValidation;
import model.Injection;
import model.Medicine;
import model.Supplier;
import model.Syrup;
import model.Tablet;

public class FileUtil {
    public static final String DATA_FILE = "data/medicines.csv";
    public static final String REPORT_FILE = "data/report.txt";
    private static final String HEADER =
    "type,id,name,price,qty,expiry,threshold,supplierName,supplierPhone,extra";

    public static void saveReport(String text) throws IOException {
        new File("data").mkdirs();
        FileWriter writer = new FileWriter(REPORT_FILE);
        try {
            writer.write(text);
        } finally {
            writer.close();
        }
    }

    // pehle temp file me likho, phir asli file se replace karo taaki data corrupt na ho
    public static void saveMedicines(ArrayList<Medicine> list) throws IOException {
        File temporary = new File(DATA_FILE + ".tmp");
        new File("data").mkdirs();
        FileWriter writer = new FileWriter(temporary);
```

```
        writer.write(HEADER + "\n");
        for (Medicine medicine : list) {
            writer.write(medicine.toCsv() + "\n");
        }
        writer.close();
        File target = new File(DATA_FILE);
        target.delete();
        if (!temporary.renameTo(target)) {
            throw new IOException("Could not replace the medicines file");
        }
    }

    public static ArrayList<Medicine> loadMedicines() throws IOException {
        return loadMedicines(DATA_FILE);
    }

    static ArrayList<Medicine> loadMedicines(String filename) throws IOException {
        ArrayList<Medicine> list = new ArrayList<Medicine>();
        HashSet<String> ids = new HashSet<String>();
        BufferedReader reader = new BufferedReader(new FileReader(filename));
        try {
            if (!HEADER.equals(reader.readLine())) {
                throw new IOException("Invalid CSV header in " + filename);
            }
            String line;
            while ((line = reader.readLine()) != null) {
                if (!line.trim().isEmpty()) {
                    Medicine medicine = readMedicine(line);
                    if (!ids.add(medicine.getId())) {
                        throw new IOException("Duplicate medicine ID: " + medicine.getId());
                    }
                    list.add(medicine);
                }
            }
        } finally {
            reader.close();
        }
        return list;
    }

    private static Medicine readMedicine(String line) throws IOException {
        String[] fields = line.split(",", -1);
        if (fields.length != 10) {
            throw new IOException("Each medicine row must have 10 columns");
        }
        try {
            Medicine medicine;
            try {
                double price = Double.parseDouble(fields[3]);
                int quantity = Integer.parseInt(fields[4]);
                LocalDate expiry = LocalDate.parse(fields[5]);
                int limit = Integer.parseInt(fields[6]);
                int extra = Integer.parseInt(fields[9]);
                Supplier supplier = new Supplier(fields[7], fields[8]);
                if (fields[0].equals("Tablet")) {
                    medicine = new Tablet(fields[1], fields[2], price, quantity, expiry, limit, supplier,
    extra);
                } else if (fields[0].equals("Syrup")) {
                    medicine = new Syrup(fields[1], fields[2], price, quantity, expiry, limit, supplier,
    extra);
                } else if (fields[0].equals("Injection")) {
                    medicine = new Injection(fields[1], fields[2], price, quantity, expiry, limit, supplier,
    extra);
                } else {
                    throw new IOException("Unknown medicine type: " + fields[0]);
                }
            } catch (NumberFormatException | DateTimeParseException e) {
                throw new IOException("Invalid number or date in medicine row", e);
            }
            MedicineValidation.check(medicine);
            return medicine;
        } catch (InvalidInputException e) {
            throw new IOException(e.getMessage(), e);
        }
    }
}
```

src/utils/InventoryFileLock.java

```java
package utils;

import java.io.File;
import java.io.IOException;

public class InventoryFileLock implements AutoCloseable {
    private File lockFile = new File("data/inventory.lock");
    private boolean locked = false;

    // jab tak lock file hai, koi aur program CSV change nahi karega
    public static InventoryFileLock acquire() throws IOException {
        InventoryFileLock lock = new InventoryFileLock();
        new File("data").mkdirs();
        for (int i = 0; i < 50; i++) {
            if (lock.lockFile.createNewFile()) {
                lock.locked = true;
                return lock;
            }
            try {
                Thread.sleep(100);
            } catch (InterruptedException e) {
                break;
            }
        }
        throw new IOException("Inventory file is busy, try again after some time");
    }

    public void close() throws IOException {
        if (locked) {
            lockFile.delete();
            locked = false;
        }
    }
}
```

src/utils/SortUtil.java

```java
package utils;

import java.util.ArrayList;
import java.util.List;
import model.Medicine;

public class SortUtil {

    public static void sortByExpiry(ArrayList<Medicine> list) {
        list.sort((m1, m2) -> m1.getExpiryDate().compareTo(m2.getExpiryDate()));
    }

    public static ArrayList<Medicine> filterLowStock(ArrayList<Medicine> list) {
        ArrayList<Medicine> low = new ArrayList<Medicine>();
        for (Medicine m : list) {
            if (m.isLowStock()) {
                low.add(m);
            }
        }
        return low;
    }

    public static <T> void printAll(List<T> list) {
        for (T item : list) {
            System.out.println(item);
        }
    }

    public static void printReportLines(ArrayList<Medicine> list) {
        for (Medicine m : list) {
            System.out.println(m.getReportLine());
        }
    }
}
```

Package network

TCP inventory server and counter client

src/network/CounterClient.java

```
1  package network;
2
3  import java.io.BufferedReader;
4  import java.io.IOException;
5  import java.io.InputStreamReader;
6  import java.io.PrintWriter;
7  import java.net.Socket;
8
9  public class CounterClient {
10
11     private static final String SERVER = "localhost";
12     private static final int PORT = 5555;
13
14     public static void main(String[] args) {
15         if (args.length != 2) {
16             System.out.println("Usage: java -cp out network.CounterClient <id> <qty>");
17             return;
18         }
19         try {
20             int qty = Integer.parseInt(args[1]);
21             System.out.println(sendSell(args[0], qty));
22         } catch (NumberFormatException e) {
23             System.out.println("ERROR quantity must be a number");
24         }
25     }
26
27     public static String sendSell(String id, int qty) {
28         if (id.trim().isEmpty() || id.contains(" ") || id.contains("\n")
29                 || id.contains("\r") || id.contains("\t") || qty <= 0) {
30             return "ERROR enter a valid id and positive quantity";
31         }
32         Socket socket = null;
33         BufferedReader in = null;
34         PrintWriter out = null;
35         try {
36             socket = new Socket(SERVER, PORT);
37             socket.setSoTimeout(5000);
38             in = new BufferedReader(new InputStreamReader(socket.getInputStream()));
39             out = new PrintWriter(socket.getOutputStream(), true);
40             out.println("SELL " + id + " " + qty);
41             return in.readLine();
42         } catch (IOException e) {
43             return "ERROR server not available";
44         } finally {
45             try {
46                 if (in != null) {
47                     in.close();
48                 }
49                 if (out != null) {
50                     out.close();
51                 }
52                 if (socket != null) {
53                     socket.close();
54                 }
55             } catch (IOException e) {
56                 System.out.println("Close problem");
57             }
58         }
59     }
60 }
```

src/network/InventoryServer.java

```
1  package network;
2
3  import java.io.BufferedReader;
4  import java.io.IOException;
5  import java.io.InputStreamReader;
6  import java.io.PrintWriter;
7  import java.net.InetAddress;
8  import java.net.ServerSocket;
9  import java.net.Socket;
10 import businesslogic.Inventory;
11 import dataprocessing.SampleData;
12
13 public class InventoryServer {
14     private static final int PORT = 5555;
15
16     public static void main(String[] args) {
```

```
        Inventory inventory = new Inventory();
        try {
            inventory.initialize(SampleData.getMedicines());
            System.out.println("Data loaded, medicines: " + inventory.getMedicines().size());
        } catch (IOException e) {
            System.out.println("Could not load data: " + e.getMessage());
            return;
        }
        try (ServerSocket server = new ServerSocket(PORT, 50, InetAddress.getByName("localhost"))) {
            System.out.println("Central Inventory Server started on port " + PORT);
            while (true) {
                handle(server.accept(), inventory);
            }
        } catch (IOException e) {
            System.out.println("Server error: " + e.getMessage());
        }
    }

    private static void handle(Socket socket, Inventory inventory) {
        try (Socket connection = socket;
                BufferedReader input = new BufferedReader(new InputStreamReader(connection.getInputStream()));
                PrintWriter output = new PrintWriter(connection.getOutputStream(), true)) {
            connection.setSoTimeout(5000);
            String request = input.readLine();
            output.println(process(request, inventory));
        } catch (IOException e) {
            System.out.println("Connection problem: " + e.getMessage());
        }
    }

    private static String process(String request, Inventory inventory) {
        try {
            if (request == null) {
                return "ERROR empty request";
            }
            String[] parts = request.trim().split("\\s+");
            if (parts.length != 3 || !parts[0].equalsIgnoreCase("SELL")) {
                return "ERROR use SELL <id> <qty>";
            }
            int remaining = inventory.getStockService().sellMedicine(parts[1], Integer.parseInt(parts[2]));
            return "OK " + remaining;
        } catch (Exception e) {
            return "ERROR " + e.getMessage();
        }
    }
}
```

Package gui

Swing screens, console menu and application entry point

src/gui/ConsoleMenu.java

```
package gui;

import java.io.IOException;
import java.util.ArrayList;
import java.util.Scanner;
import businesslogic.ExpiredMedicineException;
import businesslogic.Inventory;
import businesslogic.InvalidInputException;
import businesslogic.OutOfStockException;
import model.Medicine;
import utils.SortUtil;

public class ConsoleMenu {

    public static void start(Inventory inventory) {
        Scanner sc = new Scanner(System.in);
        boolean running = true;
        while (running) {
            System.out.println("\n1. List medicines");
            System.out.println("2. Search by name");
            System.out.println("3. Sell medicine");
            System.out.println("4. Low stock list");
            System.out.println("5. Exit");
            System.out.print("Choice: ");
            if (!sc.hasNextLine()) {
                break;
            }
            String choice = sc.nextLine();
            if (!choice.equals("5")) {
                try {
```

```
                    inventory.refresh();
                } catch (IOException e) {
                    System.out.println("Could not read stock: " + e.getMessage());
                    continue;
                }
            }
            if (choice.equals("1")) {
                SortUtil.printAll(inventory.getMedicines());
            } else if (choice.equals("2")) {
                search(sc, inventory);
            } else if (choice.equals("3")) {
                sell(sc, inventory);
            } else if (choice.equals("4")) {
                lowStock(inventory);
            } else if (choice.equals("5")) {
                running = false;
            } else {
                System.out.println("Wrong choice");
            }
        }
        System.out.println("Bye");
    }

    private static void search(Scanner sc, Inventory inventory) {
        System.out.print("Enter name: ");
        String word = sc.nextLine();
        ArrayList<Medicine> found = inventory.searchMedicine(word);
        if (found.isEmpty()) {
            System.out.println("No match");
        }
        SortUtil.printReportLines(found);
    }

    private static void sell(Scanner sc, Inventory inventory) {
        try {
            System.out.print("Medicine id: ");
            String id = sc.nextLine();
            System.out.print("Quantity: ");
            int qty = Integer.parseInt(sc.nextLine());
            inventory.getStockService().sellMedicine(id, qty);
            System.out.println("Stock updated");
        } catch (NumberFormatException e) {
            System.out.println("Quantity must be a number");
        } catch (OutOfStockException e) {
            System.out.println(e.getMessage());
        } catch (ExpiredMedicineException e) {
            System.out.println(e.getMessage());
        } catch (InvalidInputException e) {
            System.out.println(e.getMessage());
        } catch (IOException e) {
            System.out.println("Could not save stock: " + e.getMessage());
        }
    }

    private static void lowStock(Inventory inventory) {
        ArrayList<Medicine> low = SortUtil.filterLowStock(inventory.getMedicines());
        System.out.println("Low stock medicines: " + low.size());
        for (Medicine m : low) {
            System.out.println(m.getReportLine());
        }
    }
}
```

src/gui/Dashboard.java

```
package gui;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.FlowLayout;
import java.util.ArrayList;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.table.DefaultTableModel;
import businesslogic.ExpiryMonitor;
import businesslogic.Inventory;
import model.Medicine;
import usermanagement.User;
import utils.SortUtil;

```

```
21  public class Dashboard extends JFrame {
22
23      private Inventory inventory;
24      private SummaryCards cards;
25      private JLabel statusLabel;
26      DefaultTableModel model;
27      JTable table;
28
29      public Dashboard(Inventory inventory, User user) {
30          this.inventory = inventory;
31          setTitle("Medical Inventory - Dashboard");
32          setSize(1100, 650);
33          setLocationRelativeTo(null);
34          setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
35          setLayout(new BorderLayout());
36          JPanel heading = new JPanel(new BorderLayout());
37          heading.add(Ui.header("Dashboard - " + user.getRole() + " ("
38                  + user.getUsername() + ")"), BorderLayout.NORTH);
39          heading.add(new JLabel(" Kaggle catalog (Nov 2022 base prices); stock, expiry, limits and suppliers
    are demo data."),
40                  BorderLayout.SOUTH);
41          add(heading, BorderLayout.NORTH);
42
43          cards = new SummaryCards();
44          cards.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));
45
46          model = new DefaultTableModel(new String[] {"ID", "Name", "Type", "Price",
47                  "Quantity", "Expiry", "Reorder Level", "Supplier", "Status"}, 0) {
48              public boolean isCellEditable(int row, int column) {
49                  return false;
50              }
51          };
52          table = new JTable(model);
53          Ui.styleTable(table, new int[] {45, 225, 70, 75, 80, 95, 100, 115, 210});
54          table.setDefaultRenderer(Object.class, new StatusRenderer(8));
55          JPanel center = new JPanel(new BorderLayout());
56          center.setBackground(Ui.BG);
57          center.add(cards, BorderLayout.NORTH);
58          center.add(new JScrollPane(table), BorderLayout.CENTER);
59          add(center, BorderLayout.CENTER);
60
61          DashboardActions actions = new DashboardActions(this, inventory);
62          JPanel buttons = new JPanel(new FlowLayout(FlowLayout.CENTER, 8, 8));
63          buttons.setBackground(Ui.BG);
64          addButton(buttons, "Add", Ui.GREEN, actions);
65          addButton(buttons, "Update", Ui.GREEN, actions);
66          JButton deleteBtn = addButton(buttons, "Delete", Ui.RED, actions);
67          addButton(buttons, "Search", Ui.HEADER, actions);
68          addButton(buttons, "Reports", Ui.HEADER, actions);
69          addButton(buttons, "Sell", Ui.ORANGE, actions);
70          JButton backupBtn = addButton(buttons, "Backup", Ui.HEADER, actions);
71          JButton restoreBtn = addButton(buttons, "Restore", Ui.HEADER, actions);
72          addButton(buttons, "Refresh", Ui.GREEN, actions);
73
74          if (!user.isAdmin()) {
75              deleteBtn.setEnabled(false);
76              backupBtn.setEnabled(false);
77              restoreBtn.setEnabled(false);
78          }
79
80          statusLabel = new JLabel(" Ready");
81          statusLabel.setOpaque(true);
82          statusLabel.setBackground(Ui.BG);
83          JPanel south = new JPanel(new BorderLayout());
84          south.setBackground(Ui.BG);
85          south.add(buttons, BorderLayout.NORTH);
86          south.add(statusLabel, BorderLayout.SOUTH);
87          add(south, BorderLayout.SOUTH);
88
89          refreshTable();
90          startMonitor();
91      }
92
93      private JButton addButton(JPanel panel, String text, Color color, DashboardActions actions) {
94          JButton b = Ui.button(text, color);
95          b.addActionListener(actions);
96          panel.add(b);
97          return b;
98      }
99
100     void refreshTable() {
101         model.setRowCount(0);
102         ArrayList<Medicine> list = inventory.getMedicines();
103         SortUtil.sortByExpiry(list);
```

```
        for (Medicine m : list) {
            model.addRow(new Object[] {
                m.getId(), m.getName(), m.getClass().getSimpleName(),
                String.format("%.2f", m.calculatePrice()), m.getQty(),
                m.getExpiryDate().toString(), m.getThreshold(),
                m.getSupplier().getName(), m.getStatus()
            });
        }
        cards.refresh(inventory);
    }

    private void startMonitor() {
        Runnable refreshView = new Runnable() {
            public void run() {
                refreshTable();
            }
        };
        ExpiryMonitor monitor = new ExpiryMonitor(inventory, statusLabel, refreshView);
        Thread thread = new Thread(monitor);
        thread.setDaemon(true);
        thread.setPriority(Thread.MIN_PRIORITY);
        thread.start();
    }
}
```

src/gui/DashboardActions.java

```
package gui;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.IOException;
import javax.swing.JOptionPane;
import businesslogic.Inventory;
import businesslogic.InvalidInputException;
import model.Medicine;
import utils.BackupUtil;

public class DashboardActions implements ActionListener {
    private Dashboard dashboard;
    private Inventory inventory;

    public DashboardActions(Dashboard dashboard, Inventory inventory) {
        this.dashboard = dashboard;
        this.inventory = inventory;
    }

    public void actionPerformed(ActionEvent event) {
        try {
            String command = event.getActionCommand();
            if (command.equals("Restore")) {
                restoreBackup();
                return;
            }
            if (command.equals("Backup")) {
                BackupUtil.backup();
                JOptionPane.showMessageDialog(dashboard, "Backup saved");
                return;
            }
            inventory.refresh();
            if (command.equals("Add")) {
                new MedicineForm(inventory, null, dashboard).setVisible(true);
            } else if (command.equals("Update")) {
                editSelected();
            } else if (command.equals("Delete")) {
                deleteSelected();
            } else if (command.equals("Search")) {
                new SearchScreen(inventory).setVisible(true);
            } else if (command.equals("Reports")) {
                new ReportScreen(inventory).setVisible(true);
            } else if (command.equals("Sell")) {
                new SellDialog(inventory, dashboard).setVisible(true);
            } else if (command.equals("Backup")) {
                BackupUtil.backup();
                JOptionPane.showMessageDialog(dashboard, "Backup saved");
            } else if (command.equals("Restore")) {
                restoreBackup();
            } else if (command.equals("Refresh")) {
                dashboard.refreshTable();
            }
        } catch (IOException | InvalidInputException e) {
            JOptionPane.showMessageDialog(dashboard, e.getMessage());
        }
    }

```

```
    private String selectedId() {
        int row = dashboard.table.getSelectedRow();
        if (row == -1) {
            JOptionPane.showMessageDialog(dashboard, "Select a medicine first");
            return null;
        }
        return dashboard.model.getValueAt(row, 0).toString();
    }

    private void editSelected() {
        String id = selectedId();
        if (id == null) {
            return;
        }
        Medicine medicine = inventory.getById(id);
        if (medicine == null) {
            JOptionPane.showMessageDialog(dashboard, "Medicine no longer exists. Refresh the table");
        } else {
            new MedicineForm(inventory, medicine, dashboard).setVisible(true);
        }
    }

    private void deleteSelected() throws IOException, InvalidInputException {
        String id = selectedId();
        if (id == null) {
            return;
        }
        int answer = JOptionPane.showConfirmDialog(dashboard, "Delete medicine " + id + "?");
        if (answer == JOptionPane.YES_OPTION) {
            inventory.deleteMedicine(id);
            dashboard.refreshTable();
        }
    }

    private void restoreBackup() throws IOException {
        int answer = JOptionPane.showConfirmDialog(dashboard, "Replace current data with the backup?");
        if (answer == JOptionPane.YES_OPTION) {
            BackupUtil.restore();
            inventory.refresh();
            dashboard.refreshTable();
            JOptionPane.showMessageDialog(dashboard, "Data restored from backup");
        }
    }
}
```

src/gui/LoginScreen.java

```
package gui;

import java.awt.BorderLayout;
import java.awt.GridLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JOptionPane;
import javax.swing.JPanel;
import javax.swing.JPasswordField;
import javax.swing.JTextField;
import javax.swing.SwingConstants;
import businesslogic.Inventory;
import usermanagement.LoginCheck;
import usermanagement.User;

public class LoginScreen extends JFrame implements ActionListener {

    private JTextField userField;
    private JPasswordField passField;
    private Inventory inventory;

    public LoginScreen(Inventory inventory) {
        this.inventory = inventory;
        setTitle("Medical Inventory - Login");
        setSize(400, 280);
        setLocationRelativeTo(null);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        add(Ui.header("Intelligent Medical Inventory"), BorderLayout.NORTH);

        JPanel form = new JPanel(new GridLayout(2, 2, 8, 8));
        form.setBackground(Ui.BG);
        form.setBorder(BorderFactory.createEmptyBorder(20, 30, 10, 30));
```

```
        form.add(new JLabel("Username:"));
        userField = new JTextField();
        form.add(userField);
        form.add(new JLabel("Password:"));
        passField = new JPasswordField();
        form.add(passField);
        add(form, BorderLayout.CENTER);

        JButton loginBtn = Ui.button("Login", Ui.HEADER);
        loginBtn.addActionListener(this);
        JLabel hint = new JLabel("admin/admin123 or pharma/pharma123", SwingConstants.CENTER);
        JPanel bottom = new JPanel(new GridLayout(2, 1, 5, 5));
        bottom.setBackground(Ui.BG);
        bottom.setBorder(BorderFactory.createEmptyBorder(0, 30, 20, 30));
        bottom.add(loginBtn);
        bottom.add(hint);
        add(bottom, BorderLayout.SOUTH);
    }

    public void actionPerformed(ActionEvent e) {
        String name = userField.getText();
        String pass = new String(passField.getPassword());
        User user = LoginCheck.check(name, pass);
        if (user == null) {
            JOptionPane.showMessageDialog(this, "Wrong username or password");
            return;
        }
        dispose();
        new Dashboard(inventory, user).setVisible(true);
    }
}
```

src/gui/Main.java

```
package gui;

import java.io.IOException;
import javax.swing.SwingUtilities;
import businesslogic.Inventory;
import dataprocessing.SampleData;
import model.Hospital;

public class Main {
    public static void main(String[] args) {
        Hospital hospital = new Hospital("NIET Hospital");
        final Inventory inventory = hospital.getInventory();
        try {
            inventory.initialize(SampleData.getMedicines());
            System.out.println(hospital.getName() + " loaded medicines: "
                    + inventory.getMedicines().size());
        } catch (IOException e) {
            System.out.println("Data file problem: " + e.getMessage());
            return;
        }
        if (args.length > 0 && args[0].equals("--console")) {
            ConsoleMenu.start(inventory);
        } else {
            Ui.look();
            SwingUtilities.invokeLater(new Runnable() {
                public void run() {
                    new LoginScreen(inventory).setVisible(true);
                }
            });
        }
    }
}
```

src/gui/MedicineForm.java

```
package gui;

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.GridLayout;
import java.time.LocalDate;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;
import businesslogic.Inventory;
import model.Medicine;

```

```
public class MedicineForm extends JFrame {
    Inventory inventory;
    Dashboard dashboard;
    boolean updateMode;
    JTextField idField = new JTextField();
    JTextField nameField = new JTextField();
    JTextField priceField = new JTextField();
    JTextField qtyField = new JTextField();
    JTextField expiryField = new JTextField(LocalDate.now().plusYears(2).toString());
    JTextField limitField = new JTextField();
    JTextField supplierField = new JTextField();
    JTextField phoneField = new JTextField();
    JTextField extraField = new JTextField();
    JComboBox<String> typeBox = new JComboBox<String>(new String[] {"Tablet", "Syrup", "Injection"});

    public MedicineForm(Inventory inventory, Medicine medicine, Dashboard dashboard) {
        this.inventory = inventory;
        this.dashboard = dashboard;
        updateMode = medicine != null;
        String title = medicine == null ? "Add Medicine" : "Update Medicine";
        setTitle(title);
        setSize(440, 520);
        setLocationRelativeTo(dashboard);
        setLayout(new BorderLayout());
        add(Ui.header(title), BorderLayout.NORTH);
        JPanel form = new JPanel(new GridLayout(10, 2, 6, 6));
        form.setBackground(Ui.BG);
        form.setBorder(BorderFactory.createEmptyBorder(15, 20, 10, 20));
        addField(form, "Medicine ID:", idField);
        addField(form, "Name:", nameField);
        form.add(new JLabel("Type:"));
        form.add(typeBox);
        addField(form, "Base price (Rs):", priceField);
        addField(form, "Quantity:", qtyField);
        addField(form, "Expiry (yyyy-MM-dd):", expiryField);
        addField(form, "Reorder level:", limitField);
        addField(form, "Supplier name:", supplierField);
        addField(form, "Supplier phone:", phoneField);
        addField(form, "Extra (strip/ml/mg):", extraField);
        add(form, BorderLayout.CENTER);
        MedicineFormActions actions = new MedicineFormActions(this);
        JButton saveBtn = Ui.button("Save", Ui.GREEN);
        saveBtn.addActionListener(actions);
        JButton closeBtn = Ui.button("Close", Ui.HEADER);
        closeBtn.addActionListener(actions);
        JPanel bottom = new JPanel(new FlowLayout());
        bottom.setBackground(Ui.BG);
        bottom.add(saveBtn);
        bottom.add(closeBtn);
        add(bottom, BorderLayout.SOUTH);
        if (medicine != null) {
            loadMedicine(medicine);
        }
    }

    private void addField(JPanel form, String label, JTextField field) {
        form.add(new JLabel(label));
        form.add(field);
    }

    private void loadMedicine(Medicine medicine) {
        idField.setText(medicine.getId());
        idField.setEditable(false);
        nameField.setText(medicine.getName());
        typeBox.setSelectedItem(medicine.getClass().getSimpleName());
        priceField.setText(String.valueOf(medicine.getPrice()));
        qtyField.setText(String.valueOf(medicine.getQty()));
        expiryField.setText(medicine.getExpiryDate().toString());
        limitField.setText(String.valueOf(medicine.getThreshold()));
        supplierField.setText(medicine.getSupplier().getName());
        phoneField.setText(medicine.getSupplier().getPhone());
        extraField.setText(String.valueOf(medicine.getExtra()));
    }
}
```

src/gui/MedicineFormActions.java

```java
package gui;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.IOException;
import java.time.LocalDate;
import java.time.format.DateTimeParseException;
import javax.swing.JOptionPane;
import businesslogic.DuplicateMedicineException;
import businesslogic.InvalidInputException;
import model.Injection;
import model.Medicine;
import model.Supplier;
import model.Syrup;
import model.Tablet;

public class MedicineFormActions implements ActionListener {
    private MedicineForm form;

    public MedicineFormActions(MedicineForm form) {
        this.form = form;
    }

    public void actionPerformed(ActionEvent event) {
        if (event.getActionCommand().equals("Close")) {
            form.dispose();
            return;
        }
        try {
            String id = form.idField.getText().trim();
            String name = form.nameField.getText().trim();
            double price = Double.parseDouble(form.priceField.getText().trim());
            int quantity = Integer.parseInt(form.qtyField.getText().trim());
            LocalDate expiry = LocalDate.parse(form.expiryField.getText().trim());
            int limit = Integer.parseInt(form.limitField.getText().trim());
            int extra = Integer.parseInt(form.extraField.getText().trim());
            Supplier supplier = new Supplier(form.supplierField.getText().trim(),
                    form.phoneField.getText().trim());
            String type = form.typeBox.getSelectedItem().toString();
            Medicine medicine;
            if (type.equals("Tablet")) {
                medicine = new Tablet(id, name, price, quantity, expiry, limit, supplier, extra);
            } else if (type.equals("Syrup")) {
                medicine = new Syrup(id, name, price, quantity, expiry, limit, supplier, extra);
            } else {
                medicine = new Injection(id, name, price, quantity, expiry, limit, supplier, extra);
            }
            String message;
            if (!form.updateMode) {
                form.inventory.addMedicine(medicine);
                message = "Medicine added";
            } else {
                form.inventory.updateMedicine(medicine);
                message = "Medicine updated";
            }
            form.dashboard.refreshTable();
            JOptionPane.showMessageDialog(form, message);
            form.dispose();
        } catch (NumberFormatException e) {
            JOptionPane.showMessageDialog(form, "Price, quantity, reorder level and extra must be numbers");
        } catch (DateTimeParseException e) {
            JOptionPane.showMessageDialog(form, "Expiry must be yyyy-MM-dd");
        } catch (InvalidInputException | DuplicateMedicineException e) {
            JOptionPane.showMessageDialog(form, e.getMessage());
        } catch (IOException e) {
            JOptionPane.showMessageDialog(form, "Could not save medicine: " + e.getMessage());
        }
    }
}
```

src/gui/ReportScreen.java

```java
package gui;

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.Font;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.IOException;
import utils.FileUtil;
import javax.swing.BorderFactory;
```

```
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTextArea;
import businesslogic.Inventory;
import dataprocessing.SalesData;
import reporting.AnalyticsReport;
import reporting.ReportMaker;

public class ReportScreen extends JFrame implements ActionListener {

    private Inventory inventory;
    private JComboBox<String> typeBox = new JComboBox<String>(new String[] {
            "Stock Report", "Expiry Report", "Reorder Report", "Fast Moving",
            "Supplier List", "Activity Log", "Weekly Sales"});
    private JTextArea area = new JTextArea();
    private JLabel msgLabel = new JLabel("Choose a report and press Generate");

    public ReportScreen(Inventory inventory) {
        this.inventory = inventory;
        setTitle("Reports and Analytics");
        setSize(700, 520);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        add(Ui.header("Reports and Analytics"), BorderLayout.NORTH);

        JPanel top = new JPanel(new FlowLayout());
        top.setBackground(Ui.BG);
        top.add(new JLabel("Report:"));
        top.add(typeBox);
        JButton generateBtn = Ui.button("Generate", Ui.HEADER);
        generateBtn.addActionListener(this);
        top.add(generateBtn);
        JButton saveBtn = Ui.button("Save Text", Ui.GREEN);
        saveBtn.addActionListener(this);
        top.add(saveBtn);
        JButton closeBtn = Ui.button("Close", Ui.RED);
        closeBtn.addActionListener(this);
        top.add(closeBtn);

        area.setFont(new Font("Monospaced", Font.PLAIN, 13));
        area.setEditable(false);

        JPanel center = new JPanel(new BorderLayout());
        center.setBackground(Ui.BG);
        center.add(top, BorderLayout.NORTH);
        center.add(new JScrollPane(area), BorderLayout.CENTER);
        add(center, BorderLayout.CENTER);

        msgLabel.setBorder(BorderFactory.createEmptyBorder(5, 10, 5, 10));
        add(msgLabel, BorderLayout.SOUTH);
    }

    public void actionPerformed(ActionEvent e) {
        if (e.getActionCommand().equals("Close")) {
            dispose();
        } else {
            generate();
            if (e.getActionCommand().equals("Save Text")) {
                try {
                    FileUtil.saveReport(area.getText());
                    msgLabel.setText("Saved to " + FileUtil.REPORT_FILE);
                } catch (IOException ex) {
                    msgLabel.setText("Could not save report: " + ex.getMessage());
                }
            }
        }
    }

    private void generate() {
        String type = typeBox.getSelectedItem().toString();
        if (type.equals("Stock Report")) {
            area.setText(ReportMaker.stockReport(inventory));
        } else if (type.equals("Expiry Report")) {
            area.setText(ReportMaker.expiryReport(inventory));
        } else if (type.equals("Reorder Report")) {
            area.setText(ReportMaker.reorderReport(inventory));
        } else if (type.equals("Fast Moving")) {
            area.setText(AnalyticsReport.fastMovingReport(inventory));
        } else if (type.equals("Supplier List")) {
```

```
            area.setText(AnalyticsReport.supplierReport(inventory));
        } else if (type.equals("Activity Log")) {
            area.setText(AnalyticsReport.activityReport(inventory));
        } else {
            area.setText(new SalesData().getReport());
        }
        msgLabel.setText("Report generated");
    }
}
```

src/gui/SearchScreen.java

```
package gui;

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.util.ArrayList;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JComboBox;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JScrollPane;
import javax.swing.JTable;
import javax.swing.JTextField;
import javax.swing.table.DefaultTableModel;
import businesslogic.Inventory;
import model.Medicine;

public class SearchScreen extends JFrame implements ActionListener {

    private Inventory inventory;
    private JTextField keyField = new JTextField(18);
    private JComboBox<String> modeBox = new JComboBox<String>(
            new String[] {"By Name", "By ID"});
    private DefaultTableModel model;
    private JLabel resultLabel = new JLabel("Type a keyword and press Search");

    public SearchScreen(Inventory inventory) {
        this.inventory = inventory;
        setTitle("Search Medicine");
        setSize(800, 400);
        setLocationRelativeTo(null);
        setLayout(new BorderLayout());

        add(Ui.header("Search Medicine"), BorderLayout.NORTH);

        JPanel top = new JPanel(new FlowLayout());
        top.setBackground(Ui.BG);
        top.add(modeBox);
        top.add(keyField);
        JButton searchBtn = Ui.button("Search", Ui.HEADER);
        searchBtn.addActionListener(this);
        top.add(searchBtn);

        model = new DefaultTableModel(new String[] {"ID", "Name", "Quantity", "Status"}, 0) {
            public boolean isCellEditable(int row, int column) {
                return false;
            }
        };
        JTable table = new JTable(model);
        Ui.styleTable(table, new int[] {80, 340, 90, 270});
        table.setDefaultRenderer(Object.class, new StatusRenderer(3));

        JPanel center = new JPanel(new BorderLayout());
        center.setBackground(Ui.BG);
        center.add(top, BorderLayout.NORTH);
        center.add(new JScrollPane(table), BorderLayout.CENTER);
        add(center, BorderLayout.CENTER);

        resultLabel.setBorder(BorderFactory.createEmptyBorder(5, 10, 5, 10));
        add(resultLabel, BorderLayout.SOUTH);
    }

    public void actionPerformed(ActionEvent e) {
        model.setRowCount(0);
        String text = keyField.getText().trim();
        if (text.isEmpty()) {
            resultLabel.setText("Type something to search");
            return;
        }
        try {
```

```
            if (modeBox.getSelectedItem().toString().equals("By ID")) {
                int number = Integer.parseInt(text);
                Medicine m = inventory.searchMedicine(number);
                if (m == null) {
                    resultLabel.setText("No medicine with id " + number);
                } else {
                    addRow(m);
                    resultLabel.setText("Found 1 medicine");
                }
            } else {
                ArrayList<Medicine> found = inventory.searchMedicine(text);
                for (Medicine m : found) {
                    addRow(m);
                }
                resultLabel.setText("Found " + found.size() + " medicine(s)");
            }
        } catch (NumberFormatException e2) {
            resultLabel.setText("ID must be a number");
        }
    }

    private void addRow(Medicine m) {
        model.addRow(new Object[] {m.getId(), m.getName(), m.getQty(), m.getStatus()});
    }
}
```

src/gui/SellActions.java

```
package gui;

import java.awt.event.ActionEvent;
import java.awt.event.ActionListener;
import java.io.IOException;
import javax.swing.SwingUtilities;
import businesslogic.ExpiredMedicineException;
import businesslogic.InvalidInputException;
import businesslogic.OutOfStockException;
import network.CounterClient;

public class SellActions implements ActionListener {
    private SellDialog dialog;

    public SellActions(SellDialog dialog) {
        this.dialog = dialog;
    }

    public void actionPerformed(ActionEvent event) {
        String command = event.getActionCommand();
        if (command.equals("Close")) {
            dialog.dispose();
            return;
        }
        try {
            String id = dialog.idField.getText().trim();
            int quantity = Integer.parseInt(dialog.qtyField.getText().trim());
            if (command.equals("Sell")) {
                dialog.inventory.getStockService().sellMedicine(id, quantity);
                dialog.msgLabel.setText("Stock updated");
                dialog.dashboard.refreshTable();
            } else {
                sendToServer(id, quantity);
            }
        } catch (NumberFormatException e) {
            dialog.msgLabel.setText("Quantity must be a number");
        } catch (OutOfStockException | ExpiredMedicineException | InvalidInputException e) {
            dialog.msgLabel.setText(e.getMessage());
        } catch (IOException e) {
            dialog.msgLabel.setText("Could not save stock: " + e.getMessage());
        }
    }

    // network call alag thread pe chalta hai, warna window freeze ho jati hai
    private void sendToServer(final String id, final int quantity) {
        dialog.sellBtn.setEnabled(false);
        dialog.serverBtn.setEnabled(false);
        dialog.msgLabel.setText("Sending request...");
        Thread counter = new Thread(new Runnable() {
            public void run() {
                String reply = CounterClient.sendSell(id, quantity);
                String message = reply == null ? "No reply from server" : reply;
                if (reply != null && reply.startsWith("OK ")) {
                    dialog.inventory.getStockService().recordSale(id, quantity);
                    try {
                        dialog.inventory.refresh();
                        message = "Sold on server, remaining quantity " + reply.substring(3);
```

```
                    } catch (IOException e) {
                        message = "Sale completed, but refresh failed: " + e.getMessage();
                    }
                }
                final String result = message;
                SwingUtilities.invokeLater(new Runnable() {
                    public void run() {
                        dialog.msgLabel.setText(result);
                        dialog.sellBtn.setEnabled(true);
                        dialog.serverBtn.setEnabled(true);
                        dialog.dashboard.refreshTable();
                    }
                });
            }
        });
        counter.start();
    }
}
```

src/gui/SellDialog.java

```
package gui;

import java.awt.BorderLayout;
import java.awt.FlowLayout;
import java.awt.GridLayout;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JFrame;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.JTextField;
import businesslogic.Inventory;

public class SellDialog extends JFrame {
    Inventory inventory;
    Dashboard dashboard;
    JTextField idField = new JTextField();
    JTextField qtyField = new JTextField();
    JLabel msgLabel = new JLabel("Enter ID and quantity");
    JButton sellBtn = Ui.button("Sell", Ui.GREEN);
    JButton serverBtn = Ui.button("Send to Server", Ui.HEADER);

    public SellDialog(Inventory inventory, Dashboard dashboard) {
        this.inventory = inventory;
        this.dashboard = dashboard;
        setTitle("Sell Stock");
        setSize(400, 280);
        setLocationRelativeTo(dashboard);
        setLayout(new BorderLayout());
        add(Ui.header("Sell Stock"), BorderLayout.NORTH);
        JPanel form = new JPanel(new GridLayout(2, 2, 8, 8));
        form.setBackground(Ui.BG);
        form.setBorder(BorderFactory.createEmptyBorder(20, 30, 10, 30));
        form.add(new JLabel("Medicine ID:"));
        form.add(idField);
        form.add(new JLabel("Quantity:"));
        form.add(qtyField);
        add(form, BorderLayout.CENTER);
        SellActions actions = new SellActions(this);
        sellBtn.addActionListener(actions);
        serverBtn.addActionListener(actions);
        JButton closeBtn = Ui.button("Close", Ui.RED);
        closeBtn.addActionListener(actions);
        JPanel buttons = new JPanel(new FlowLayout());
        buttons.setBackground(Ui.BG);
        buttons.add(sellBtn);
        buttons.add(serverBtn);
        buttons.add(closeBtn);
        msgLabel.setBorder(BorderFactory.createEmptyBorder(5, 30, 5, 30));
        JPanel bottom = new JPanel(new BorderLayout());
        bottom.setBackground(Ui.BG);
        bottom.add(buttons, BorderLayout.NORTH);
        bottom.add(msgLabel, BorderLayout.SOUTH);
        add(bottom, BorderLayout.SOUTH);
    }
}
```

src/gui/StatusRenderer.java

```
package gui;

import java.awt.Color;
import java.awt.Component;
import javax.swing.JTable;
import javax.swing.table.DefaultTableCellRenderer;

public class StatusRenderer extends DefaultTableCellRenderer {

    private int statusColumn;

    public StatusRenderer(int statusColumn) {
        this.statusColumn = statusColumn;
    }

    public Component getTableCellRendererComponent(JTable table, Object value,
            boolean selected, boolean focused, int row, int column) {
        Component c = super.getTableCellRendererComponent(table, value,
                selected, focused, row, column);
        String status = table.getValueAt(row, statusColumn).toString();
        if (!selected) {
            if (status.equals("EXPIRED")) {
                c.setBackground(new Color(255, 210, 210));
            } else if (status.equals("OK")) {
                c.setBackground(new Color(215, 245, 215));
            } else {
                c.setBackground(new Color(255, 235, 190));
            }
        }
        return c;
    }
}
```

src/gui/SummaryCards.java

```
package gui;

import java.awt.BorderLayout;
import java.awt.Color;
import java.awt.GridLayout;
import javax.swing.BorderFactory;
import javax.swing.JLabel;
import javax.swing.JPanel;
import javax.swing.SwingConstants;
import businesslogic.Inventory;
import model.Medicine;

public class SummaryCards extends JPanel {

    private JLabel totalValue;
    private JLabel lowValue;
    private JLabel expiringValue;
    private JLabel expiredValue;

    public SummaryCards() {
        setLayout(new GridLayout(1, 4, 10, 10));
        setBackground(Ui.BG);
        totalValue = addCard("Total Medicines", Ui.HEADER);
        lowValue = addCard("Low Stock", Ui.ORANGE);
        expiringValue = addCard("Expiring Soon", Ui.ORANGE);
        expiredValue = addCard("Expired", Ui.RED);
    }

    private JLabel addCard(String title, Color color) {
        JPanel card = new JPanel(new BorderLayout());
        card.setBackground(Color.WHITE);
        card.setBorder(BorderFactory.createLineBorder(color, 2));
        JLabel titleLabel = new JLabel(title, SwingConstants.CENTER);
        titleLabel.setForeground(color);
        titleLabel.setFont(Ui.NORMAL_FONT);
        JLabel valueLabel = new JLabel("0", SwingConstants.CENTER);
        valueLabel.setFont(Ui.TITLE_FONT);
        card.add(titleLabel, BorderLayout.NORTH);
        card.add(valueLabel, BorderLayout.CENTER);
        add(card);
        return valueLabel;
    }

    public void refresh(Inventory inventory) {
        int total = 0;
        int low = 0;
        int expiring = 0;
        int expired = 0;
```

```
        for (Medicine m : inventory.getMedicines()) {
            total++;
            if (m.isLowStock() && !m.isExpired()) {
                low++;
            }
            if (m.isExpiringSoon()) {
                expiring++;
            }
            if (m.isExpired()) {
                expired++;
            }
        }
        totalValue.setText(String.valueOf(total));
        lowValue.setText(String.valueOf(low));
        expiringValue.setText(String.valueOf(expiring));
        expiredValue.setText(String.valueOf(expired));
    }
}
```

src/gui/Ui.java

```
package gui;

import java.awt.Color;
import java.awt.Dimension;
import java.awt.Font;
import javax.swing.BorderFactory;
import javax.swing.JButton;
import javax.swing.JLabel;
import javax.swing.JTable;
import javax.swing.SwingConstants;
import javax.swing.UIManager;

public class Ui {

    public static final Color HEADER = new Color(0, 105, 148);
    public static final Color BG = new Color(240, 248, 250);
    public static final Color GREEN = new Color(46, 160, 67);
    public static final Color ORANGE = new Color(230, 145, 30);
    public static final Color RED = new Color(210, 55, 55);
    public static final Font TITLE_FONT = new Font("SansSerif", Font.BOLD, 20);
    public static final Font NORMAL_FONT = new Font("SansSerif", Font.PLAIN, 13);

    public static JLabel header(String text) {
        JLabel label = new JLabel(text, SwingConstants.CENTER);
        label.setOpaque(true);
        label.setBackground(HEADER);
        label.setForeground(Color.WHITE);
        label.setFont(TITLE_FONT);
        label.setPreferredSize(new Dimension(100, 50));
        return label;
    }

    public static JButton button(String text, Color color) {
        JButton b = new JButton(text);
        b.setBackground(color);
        b.setForeground(Color.WHITE);
        b.setOpaque(true);
        b.setFocusPainted(false);
        b.setFont(NORMAL_FONT);
        b.setBorder(BorderFactory.createEmptyBorder(6, 12, 6, 12));
        return b;
    }

    public static void styleTable(JTable table, int[] widths) {
        table.setRowHeight(24);
        for (int i = 0; i < widths.length; i++) {
            table.getColumnModel().getColumn(i).setPreferredWidth(widths[i]);
        }
    }

    public static void look() {
        try {
            UIManager.setLookAndFeel(UIManager.getCrossPlatformLookAndFeelClassName());
        } catch (Exception e) {
            System.out.println("Look and feel not changed");
        }
    }
}
```

Appendix: build scripts

Commands used to compile and launch the application

run.bat

```
@echo off
cd /d "%~dp0"
javac -d out src\model\*.java src\usermanagement\*.java src\businesslogic\*.java src\reporting\*.java
    src\utils\*.java src\dataprocessing\*.java src\network\*.java src\gui\*.java
if errorlevel 1 (
    echo Compilation failed. Check that the JDK is installed.
    pause
    exit /b 1
)
java -cp out gui.Main %*
pause
```

run.sh

```
#!/bin/bash
cd "$(dirname "$0")"
javac -d out src/model/*.java src/usermanagement/*.java src/businesslogic/*.java \
    src/reporting/*.java src/utils/*.java src/dataprocessing/*.java \
    src/network/*.java src/gui/*.java
if [ $? -ne 0 ]; then
    echo "Compilation failed. Check that the JDK is installed."
    exit 1
fi
java -cp out gui.Main "$@"
```